



Universidad Tecnológica de Panamá
Facultad de Ingeniería de Sistemas Computacionales
Asignatura: Herramientas Aplicadas IV
Investigación 1

Profesor: Napoleón Ibarra

Valor: 100 puntos

Nombre: Edwin Santos

Cédula: 4-828-1542

Grupo: 2IL141

Fecha: 27-3-2026

Procedimiento:

1. De manera individual, realizar la asignación.
2. Utilizando la herramienta internet, investigue cada uno de los puntos solicitados.
3. Entregar el trabajo en formato digital (PDF). Utilizar la técnica de resumen ejecutivo.

Criterios de Evaluación:

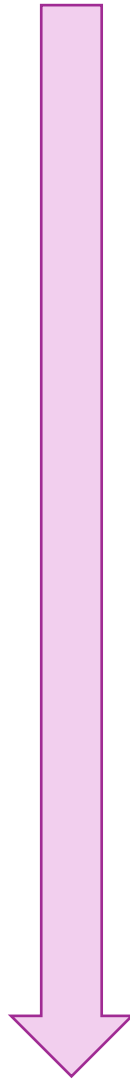
Criterios	Puntos (Mínimo=1, Máximo=5)	Porcentaje
Sustentación	1 - 5	10 %
Puntualidad	1 - 5	10 %
Creatividad (Técnica, otro)	1 - 5	10 %
Desarrollo	1 - 5	70 %

1. **Glosario ilustrado:** Apps, Aplicaciones Web para móviles, Aplicaciones nativas, Aplicaciones híbridas, IDE, SDK, SDK Manager, Carpeta Java, Carpeta Res, Carpeta Layout, Carpeta Values, Activity, Fragments, Content Provider, Broadcast Receiver, Services, XML, Android Manifest, Emulador, Dispositivo, Android, OS, Android Studio, APK, Bug, Conten Provider, Swap, UI, Widget.
2. ¿Cuál es la relevancia de un ciclo de vida de un programa en Android?
3. ¿Cuál es la mejor manera de realizar una ejecución de una Aplicación Móvil?
4. ¿Cuál es la estructura de un Proyecto en Android más importante? ¿Por qué?

RESUMEN EJECUTIVO

Qué aprendí, qué recordé, qué reafirmé	Resumen
¿Qué aprendí? Aprendí el significado de una serie de palabras relacionadas con el desarrollo móvil en Android, incluyendo los componentes principales como Activity, Services y Broadcast Receiver. También comprendí la estructura básica de un proyecto en Android y la importancia del ciclo de vida de una aplicación para gestionar correctamente los recursos. De igual forma, aprendí las diferencias entre aplicaciones nativas, híbridas y web, así como las distintas formas de ejecutar una aplicación mediante emuladores y dispositivos físicos. ¿Qué recordé? Recordé algunos conceptos básicos de programación y desarrollo de software, como la importancia de la organización del código, el uso de interfaces de usuario (UI) y la estructura de los proyectos. También reforcé conocimientos sobre el uso de herramientas de desarrollo y la necesidad de realizar pruebas constantes para garantizar el buen funcionamiento de una aplicación. ¿Qué reafirmé? Reafirmé que una buena organización del proyecto y el manejo adecuado de los recursos son fundamentales para desarrollar aplicaciones eficientes. También confirmé que el ciclo de vida de una aplicación es clave para evitar errores y mejorar la experiencia del usuario. Asimismo, reafirmé que la mejor práctica al probar aplicaciones	La presente investigación aborda una introducción a la programación de aplicaciones móviles en Android Studio. El objetivo principal es adquirir conocimientos sobre los conceptos fundamentales en el área del desarrollo de aplicaciones móviles, abarcando tanto los aspectos teóricos como estructurales que intervienen en el proceso de creación de este tipo de software. Primeramente, se presenta un glosario ilustrado con términos relevantes sobre el tema, tales como los componentes principales, estructuras del proyecto y conceptos esenciales como la interfaz de usuario (UI). Además, se analiza la relevancia del ciclo de vida de una aplicación Android, destacando la importancia de los métodos de control o “callbacks”, los cuales permiten la gestión de los diversos estados de una actividad. Se concluye que una correcta implementación de este ciclo garantiza un uso eficiente de los recursos, evita fallos y mejora la experiencia del usuario ante eventos como las interrupciones o la rotación de pantalla. Por otra parte, se estudian las distintas formas de ejecutar una aplicación móvil mediante una comparación entre el uso de emuladores y dispositivos físicos. Se determina que la mejor práctica consiste en combinar ambos métodos: el emulador para pruebas iniciales y el dispositivo físico para validar el rendimiento real y la experiencia del usuario. Igualmente, se identifica la estructura de un proyecto en Android y se resalta la importancia de partes como la carpeta de código fuente (Java), la carpeta de recursos (Res) y el archivo de configuración AndroidManifest.xml. Se establece que estos componentes, en conjunto, forman una base para el desarrollo de

es combinar el uso de emuladores y dispositivos físicos para obtener resultados más completos y reales.	aplicaciones, ya que permiten organizar la lógica, la interfaz y el funcionamiento general del sistema.
Conclusiones	
Esta investigación representó una valiosa oportunidad para adquirir una visión general de los fundamentos del desarrollo en Android. La sección del glosario fue de gran utilidad para aprender nuevos conceptos y recordar aquellos que había estudiado previamente. Finalmente, pude apreciar la importancia de la correcta organización, ejecución y gestión de los recursos para la creación de aplicaciones eficientes y funcionales.	



DESARROLLO DE LA INVESTIGACIÓN

Glosario ilustrado

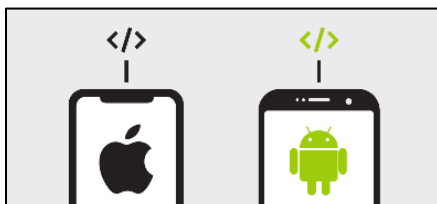
Apps: Forma abreviada de aplicaciones. Son programas diseñados para cumplir funciones específicas en dispositivos móviles, como comunicación, entretenimiento o productividad.



Aplicaciones Web para móviles: Son sitios web adaptados a pantallas móviles que se ejecutan en un navegador. No requieren instalación, pero tienen acceso limitado a funciones del dispositivo.

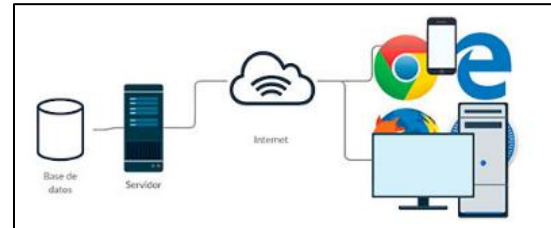


Aplicaciones nativas: Apps desarrolladas específicamente para Android usando lenguajes como Java o Kotlin. Ofrecen mejor rendimiento, acceso completo al hardware y una experiencia de usuario más optimizada.



Aplicaciones híbridas: Aplicaciones que combinan tecnologías web (HTML, CSS, JavaScript) con contenedores nativos. Permiten desarrollar una sola

base de código para múltiples plataformas, aunque con menor rendimiento que las nativas.



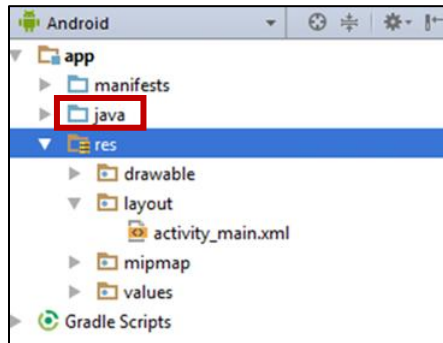
IDE (Integrated Development Environment): Software que proporciona herramientas integradas para programar, compilar y depurar aplicaciones. Un ejemplo es Android Studio.



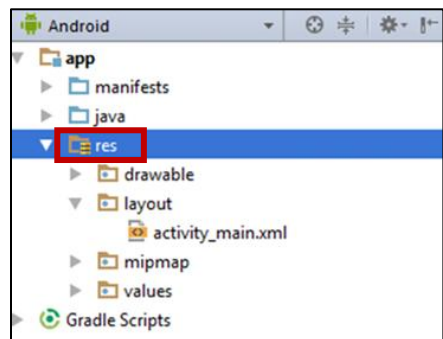
SDK (Software Development Kit): Conjunto de herramientas, librerías, documentación y ejemplos de código que permiten a los desarrolladores crear aplicaciones para una plataforma, sistema operativo o lenguaje específico. Facilita el desarrollo al evitar programar todo desde cero.



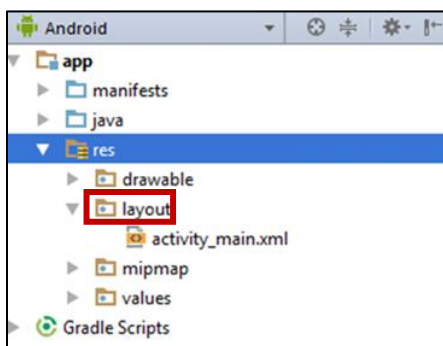
Carpeta Java: Directorio dentro del proyecto donde se almacena el código fuente (clases en Java o Kotlin). Aquí se implementa la lógica de la aplicación.



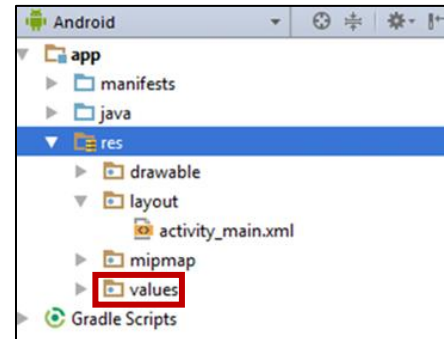
Carpeta Res: Directorio donde se almacenan recursos no ejecutables como imágenes, layouts, animaciones y textos. Facilita la organización y reutilización de elementos visuales.



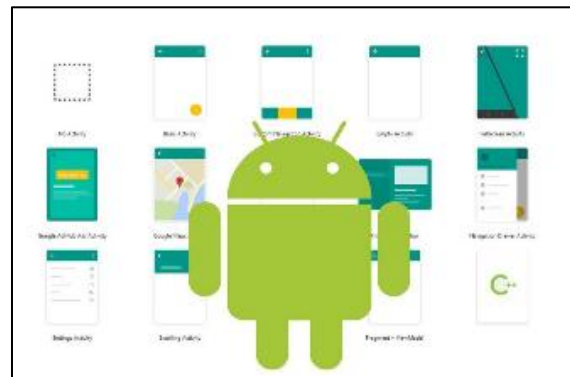
Carpeta Layout: Contiene archivos XML que definen la estructura visual de las pantallas. Permite organizar elementos de la interfaz como botones, textos e imágenes.



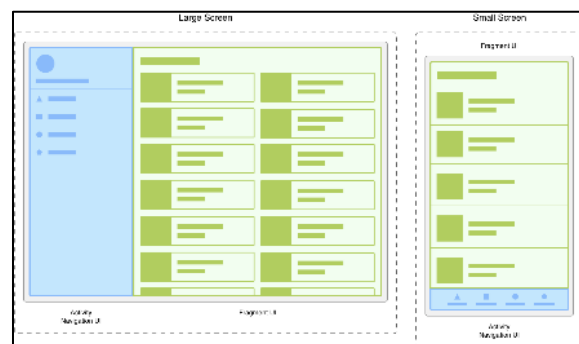
Carpeta Values: Subcarpeta de res que contiene archivos XML con valores constantes como colores, cadenas de texto (strings), estilos y temas de la aplicación.



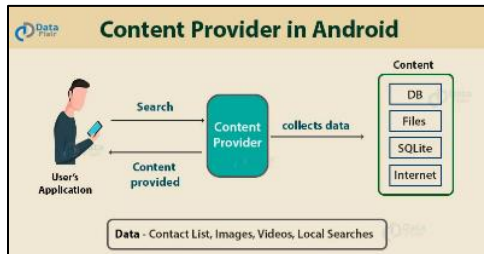
Activity: En el contexto tecnológico de Android, una Activity es el componente fundamental que representa una única pantalla con la que interactúa el usuario.



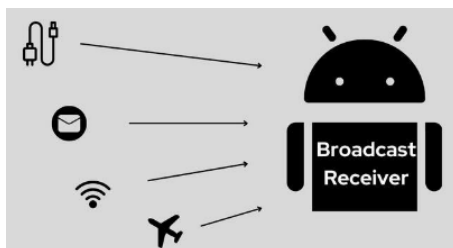
Fragments: Componentes reutilizables que representan partes de una interfaz dentro de una Activity. Permiten crear interfaces más dinámicas y adaptables.



Content Provider: Componente que gestiona el acceso a datos estructurados y permite compartirlos entre aplicaciones. Proporciona una capa segura para manejar información.



Broadcast Receiver: Componente que permite a una aplicación responder a eventos del sistema o de otras apps, como cambios de red o batería. Funciona incluso cuando la app no está activa.

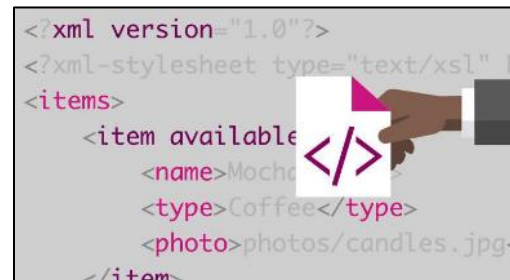


Services: Componentes que ejecutan tareas en segundo plano sin interfaz de usuario, como reproducir música o sincronizar datos mientras el usuario usa otras apps.



XML: Lenguaje de marcado utilizado en Android para definir interfaces,

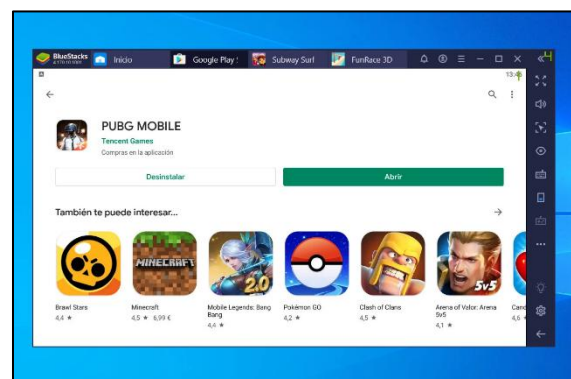
configuraciones y recursos. Permite estructurar datos de forma clara y organizada.



Android Manifest: Archivo de configuración (AndroidManifest.xml) donde se declaran los componentes de la app, permisos, actividades y configuraciones esenciales para que el sistema pueda ejecutar la aplicación correctamente.



Emulador: Herramienta que simula un dispositivo Android en la computadora. Permite probar aplicaciones sin necesidad de un dispositivo físico.



A photograph of three smartphones standing side-by-side against a solid purple background. The phone on the left is a Windows Phone, outlined with a yellow border, showing the white Windows logo on a black screen. The middle phone is a white Android phone showing the white Android robot logo on a black screen. The phone on the right is a black iPhone showing the white Apple logo on a black screen. All three phones have their status bars visible at the top, showing signal strength, battery level, and time.

A grid of eight operating system logos arranged in two rows of four. The logos are: Windows (four-pane colored flag), macOS (blue square with white lightning bolt), Linux (Tux penguin), ChromeOS (four-pane colored circle), Android (green robot), iOS (black Apple logo), UNIX (text on a grey sphere), and BSD (red dragon head).

The screenshot shows the Android Studio IDE. On the left, the 'MainActivity.kt' file is open, displaying the following Kotlin code:

```

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.TextView

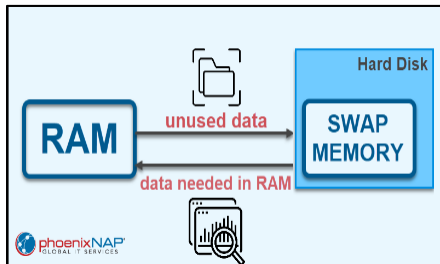
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val textView: TextView? = findViewById<TextView?>(R.id.textView)
        textView?.text = "Hello, Android Developer"
    }
}

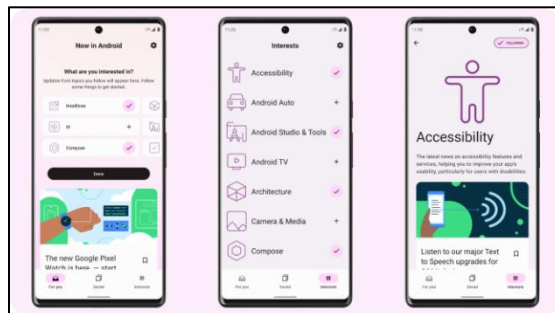
```

On the right, the 'Design' tab shows a preview of the app. The app has a green background with a white mountain silhouette and the text 'Customize your own Android bot'. The status bar at the top is green with a white Android robot icon.

Swap: Espacio en el almacenamiento que se utiliza como memoria virtual cuando la RAM se llena. Ayuda a mantener el sistema funcionando, aunque puede afectar el rendimiento.



UI: Interfaz de usuario que incluye todos los elementos visuales y de interacción. Es clave para la experiencia del usuario dentro de la aplicación.



Widget: Elemento visual de la interfaz como botones, cuadros de texto o imágenes. También se usa este término para referirse a componentes interactivos que se colocan en la pantalla principal del dispositivo.



2. ¿Cuál es la relevancia de un ciclo de vida de un programa en Android?

Durante el ciclo de vida de un programa en Android, específicamente, de una actividad, se pueden producir diversos eventos debido a la interacción del usuario con la aplicación, por ejemplo, navegar por la interfaz, pasar a otro programa y luego volver a la interfaz. Por ello, es fundamental que el comportamiento de la app esté correctamente establecido para dar una respuesta eficiente ante dichas situaciones.

Para lograr esto existen métodos denominados “callbacks” o devoluciones de llamada, que el sistema operativo llama automáticamente en el código del programa cuando ocurren eventos, como los antes mencionados. Entre estos métodos están:

- onCreate(): cuando la actividad se crea
- onStart(): cuando se vuelve visible
- onResume(): cuando el usuario puede interactuar
- onPause(): cuando el usuario deja de interactuar
- onStop(): cuando deja de ser visible

- `onDestroy()`: cuando se destruye

Estas devoluciones de llamada realizan un trabajo específico de acuerdo con un cambio de estado determinado. Debido a esto, es importante hacer el trabajo preciso en el momento adecuado y administrar las transiciones correctamente para que el programa sea más sólido y eficiente. Entre las principales ventajas de una buena implementación de las devoluciones de llamada en el ciclo de vida de un programa Android se puede mencionar, por ejemplo, que la app:

- No falla si el usuario recibe una llamada telefónica o cambia a otra app mientras usa la nuestra.
- No consume recursos valiosos del sistema cuando el usuario no la use de forma activa.
- No pierda el progreso del usuario si este abandona tu app y regresa a ella posteriormente.
- No falle ni pierda el progreso del usuario cuando se gire la pantalla entre la orientación horizontal y la vertical.

Esto es importante porque Android controla la ejecución de las aplicaciones y puede pausarlas o cerrarlas según los recursos disponibles. Por lo tanto, la relevancia del ciclo de vida radica en que permite al sistema y a la aplicación gestionar correctamente los recursos, el estado y la experiencia del usuario.

3. ¿Cuál es la mejor manera de realizar una ejecución de una Aplicación Móvil?

Las principales formas de ejecutar una aplicación móvil son mediante un dispositivo físico o un emulador. Aunque ambas maneras son válidas, la mejor opción dependerá del contexto.

El uso de un emulador, por ejemplo, en Android Studio, es ideal cuando requerimos simular distintos dispositivos, versiones del sistema y configuraciones; ya que podemos realizar pruebas rápidas, depuración inicial y observar el comportamiento de la aplicación en diferentes entornos sin la necesidad de contar con múltiples teléfonos.

Por otra parte, ejecutar la aplicación en un dispositivo físico resulta favorable cuando queremos evaluar aspectos más cercanos a la realidad como:

- Rendimiento verdadero (CPU, memoria, etc.)
- Uso de sensores (GPS, cámara, acelerómetro)
- Consumo de batería
- Experiencia de usuario

Esto evidencia que, en el desarrollo profesional, es una buena práctica adoptar un enfoque combinado; el emulador para pruebas iniciales y un dispositivo físico para pruebas finales y validación. De esta manera nos aseguramos de que la aplicación funcione correctamente antes de ser distribuida como archivo APK.

4. ¿Cuál es la estructura de un Proyecto en Android más importante? ¿Por qué?

Los proyectos de Android contienen muchos archivos relacionados con la compilación y estructura de directorios para organizar el código fuente y los recursos de la aplicación. La entidad proyecto es única y engloba todos los demás elementos; por ejemplo, los módulos, que pueden representar distintos componentes de un sistema.

En la mayoría de los casos, se trabaja con un proyecto que contendrá un solo módulo que corresponde a la aplicación principal. Los contenidos principales de este módulo, cada uno con un propósito específico, son los siguientes:

- Carpeta Java (/app/src/main/java): Esta carpeta contendrá todo el código fuente de la aplicación, clases auxiliares, etc. Inicialmente, Android Studio creará por nosotros el código básico de la pantalla (actividad o activity) principal de la aplicación, que por defecto se llamará MainActivity, y siempre bajo la estructura del paquete java definido durante la creación del proyecto.
- Carpeta Res (/app/src/main/res/): Contiene todos los ficheros de recursos necesarios para el proyecto: imágenes (drawable), diseños de interfaz (layouts), textos (values/strings.xml), colores, estilos, etc. Esta carpeta permite separar la lógica del diseño, facilitando el mantenimiento y la adaptación de la app a diferentes dispositivos.
- Archivo AndroidManifest.xml: Contiene la definición en XML de muchos de los aspectos principales de la aplicación, como por ejemplo su identificación (nombre, icono, etc.), sus componentes (pantallas, servicios, entre otros), o los permisos necesarios para su ejecución.

Este conjunto se puede considerar como la estructura más importante ya que cada parte cumple un rol esencial en el desarrollo. Además, aunque el fichero Android Manifest destaca porque es el que le dice al sistema cómo debe funcionar la aplicación, su funcionamiento depende del conjunto de elementos como el código fuente (Java) y los recursos (res), que permiten implementar la lógica y la interfaz de usuario.

Referencias

- Tipos de aplicaciones móviles: Nativas, web e híbridas – Future Space S.A. (n.d.). <https://www.futurespace.es/tipos-de-aplicaciones-moviles/>
- Souza, T., & Souza, T. (2025, May 16). ¿Qué es un SDK? Guía de SDKs Móviles, Casos de Uso y Beneficios. CleverTap - All-in-One Customer Engagement Platform. <https://clevertap.com/es/blog/que-es-un-sdk/>
- Ramírez, I. (2020, February 23). BlueStacks: guía a fondo del completo emulador de Android para PC. Xataka Android. <https://www.xatakandroid.com/aplicaciones-android/bluestacks-guia-a-fondo-completo-emulador-android-para-pc>
- Procesos y el ciclo de vida de la app. (n.d.). Android Developers. <https://developer.android.com/guide/components/activities/process-lifecycle?hl=es-419>
- Ciclo de vida de la actividad. (n.d.). Android Developers. <https://developer.android.com/guide/components/activities/activity-lifecycle?hl=es-419>
- Cómo ejecutar apps en un dispositivo de hardware. (n.d.). Android Developers. <https://developer.android.com/studio/run/device?hl=es-419>
- Compila y ejecuta tu app. (n.d.). Android Developers. <https://developer.android.com/studio/run?hl=es-419>
- Estructura de compilación de Android. (n.d.). Android Developers. <https://developer.android.com/build/android-build-structure?hl=es-419>
- Jesse. (2013, July 2). Android Studio Project Structure (v.s. Eclipse Project Structure). Stack Overflow. <https://stackoverflow.com/questions/17431838/android-studio-project-structure-v-s-eclipse-project-structure>
- Sgoliver, & Sgoliver. (2014, December 28). Estructura de un proyecto Android (Android Studio). sgoliver.net. <https://www.sgoliver.net/blog/estructura-de-un-proyecto-android-android-studio/>
- GeeksforGeeks. (2025, January 2). Android Project folder Structure. GeeksforGeeks. <https://www.geeksforgeeks.org/android/android-project-folder-structure/>